

FIAP
DEVOPS TOOLS & CLOUD COMPUTING

ORBIT API

Containerização em Nuvem de API Java Spring Boot com Docker e H2

Global Solution 2026/1 — Economia Espacial e Soluções para a Terra

Henrique Rodrigues Vespasiano — RM562917

Ruan Luca Feliciano — RM562218

Mirelly Sousa Alves — RM566299

Gabriely Bonfim Silva — RM566242

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO

2 DESCRIÇÃO DO PROJETO

3 BENEFÍCIOS PARA O NEGÓCIO

4 ARQUITETURA MACRO DA SOLUÇÃO

5 TECNOLOGIAS UTILIZADAS

6 REQUISITOS ATENDIDOS

7 EXECUÇÃO EM NUVEM COM AZURE

8 CONTEINERIZAÇÃO COM DOCKER

9 BANCO DE DADOS H2 E PERSISTÊNCIA

10 TESTES DA API E CRUD

11 EVIDÊNCIAS DOS CONTAINERS

12 LINKS DA ENTREGA

13 REMOÇÃO DOS RECURSOS EM NUVEM

14 CONCLUSÃO

REFERÊNCIAS

1 INTRODUÇÃO

Este documento apresenta a entrega da disciplina DevOps Tools & Cloud Computing, referente à containerização e execução em nuvem de uma aplicação Java Spring Boot. O projeto escolhido foi a ORBIT API, desenvolvida no contexto da Global Solution 2026/1, cujo tema envolve a economia espacial e a conexão entre a exploração espacial e problemas reais na Terra.

O objetivo da entrega foi disponibilizar a aplicação em uma Máquina Virtual Linux na Azure, utilizando Docker para containerizar a API e o banco de dados H2 em containers separados, além de garantir persistência dos dados em volume nomeado, execução em background, testes externos e documentação completa do processo.

A documentação também reúne os principais prints de evidência que comprovam o funcionamento da solução em nuvem, a execução dos containers, o acesso ao Swagger, o acesso ao H2 Console, a persistência em volume nomeado, o acesso ao terminal dos containers com usuário não-root e a remoção final dos recursos em nuvem.

2 DESCRIÇÃO DO PROJETO

A ORBIT API é a base operacional de uma plataforma de gestão de operações espaciais. A aplicação centraliza informações de missões, pessoas (tripulação), veículos, recursos e pontos de apoio, permitindo o gerenciamento dos principais dados envolvidos no planejamento e na execução de missões.

A proposta de negócio vai além de um CRUD simples. A ideia é preparar a ORBIT para evoluir como um centro de comando inteligente, com visão futura de recomendação automática de recursos por missão, análise de criticidade de operações, otimização de pontos de apoio logístico e integração de dados da exploração espacial com problemas concretos na Terra, como monitoramento ambiental e resposta a desastres.

Nesta entrega, o foco foi garantir que a API estivesse funcional, containerizada, documentada e executando em ambiente de nuvem, respeitando os requisitos técnicos definidos para a sprint, incluindo autenticação JWT, CRUD completo e persistência em múltiplas tabelas relacionadas.

3 BENEFÍCIOS PARA O NEGÓCIO

A solução traz benefícios diretos, pois permite centralizar dados importantes de missões, tripulação, veículos, recursos e pontos de apoio. Com essas informações organizadas, a operação ganha rastreabilidade, histórico das missões e uma base mais estruturada para tomada de decisão.

Com a evolução da plataforma, a solução poderá apoiar a identificação de missões críticas, recomendar a melhor alocação de recursos por missão, otimizar a distribuição dos pontos de apoio com base em localização e disponibilidade, além de conectar os dados de exploração espacial a aplicações reais na Terra, fortalecendo a proposta da economia espacial.

4 ARQUITETURA MACRO DA SOLUÇÃO

A arquitetura da solução foi construída com separação entre aplicação e banco de dados. A API Java Spring Boot roda em um container Docker dentro de uma VM Linux na Azure. O banco H2 também roda em um container separado, sendo acessado pela API via TCP, na mesma rede Docker orbit-net. Para garantir a persistência dos dados, foi utilizado um volume Docker nomeado chamado orbit-h2-data.

A aplicação é acessada externamente pela porta 8080, enquanto o console web do H2 fica disponível pela porta 8082. A porta 22 é utilizada para acesso SSH à VM e a comunicação TCP do banco ocorre pela porta 9092.

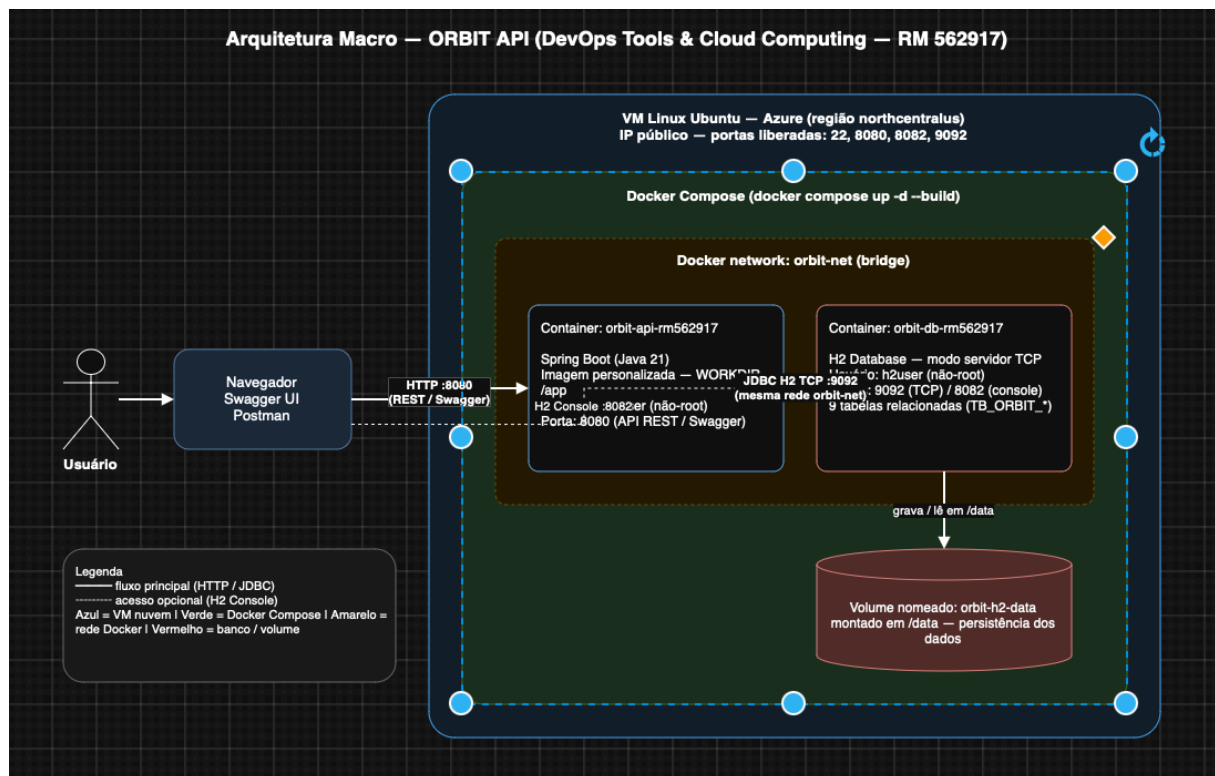


Figura 1 — Arquitetura macro da solução ORBIT (componentes em nuvem).

Usuário / Operador

|

Internet

|

IP Público da VM Azure (northcentralus)

|

VM Linux Ubuntu

|

Docker Compose

|

Container orbit-api-rm562917 – Spring Boot – Porta 8080

|

(rede Docker orbit-net – JDBC TCP 9092)

Container orbit-db-rm562917 – H2 Server – Portas 8082 / 9092

|

Volume Docker Nomeado – orbit-h2-data (/data)

5 TECNOLOGIAS UTILIZADAS

As principais tecnologias utilizadas no projeto foram Java 21, Spring Boot, Spring Data JPA, Spring Security (JWT), H2 Database, Maven, Docker, Docker Compose, Azure CLI, Máquina Virtual Linux Ubuntu na Azure, Swagger/OpenAPI, Git e GitHub.

Tecnologia	Aplicação no projeto
Java 21 e Spring Boot	Desenvolvimento da API principal (ORBIT).
Spring Data JPA	Mapeamento objeto-relacional e persistência das entidades.
Spring Security / JWT	Autenticação e proteção dos endpoints da API.
H2 Database	Banco de dados utilizado para persistência (modo servidor TCP).
Docker	Containerização da API e do banco.
Docker Compose	Orquestração dos containers da aplicação e do banco.
Azure CLI	Criação automatizada da infraestrutura em nuvem.
VM Linux Ubuntu	Ambiente de execução em nuvem.
Swagger/OpenAPI	Documentação e testes das rotas da API.
GitHub	Versionamento e disponibilização do projeto.

6 REQUISITOS ATENDIDOS

A tabela a seguir apresenta os principais requisitos da atividade e o status de atendimento no projeto.

Requisito	Status / Evidência
CRUD com GET, POST, PUT e DELETE	Atendido
Persistência em pelo menos duas tabelas relacionadas	Atendido (9 tabelas TB_ORBIT_*)
Persistência com H2	Atendido com H2 (servidor TCP)
Banco em container separado	Atendido (orbit-db-rm562917)
Imagem personalizada da aplicação	Atendido (Dockerfile multi-stage)
Dockerfile e Docker Compose	Atendido
Volume nomeado para persistência	Atendido com orbit-h2-data
WORKDIR e variável de ambiente	Atendido (/app, /h2 + env vars)
Aplicação em background	Atendido (docker compose up -d)
Containers com usuário não-root	Atendido (orbituser / h2user)
Nome dos containers com RM do representante	Atendido (rm562917)
App e banco na mesma rede Docker	Atendido (orbit-net)
VM Linux na Azure / portas abertas	Atendido (22, 8080, 8082, 9092)
Script Azure CLI	Atendido
README e How To no GitHub	Atendido
Arquitetura macro	Atendido nesta documentação e no README
Vídeo no YouTube	Atendido

Print de exclusão da VM	Atendido
-------------------------	----------

7 EXECUÇÃO EM NUVEM COM AZURE

A infraestrutura foi criada na Azure por meio de script Azure CLI. O script realiza a criação do Resource Group, provisiona uma VM Linux Ubuntu, abre as portas necessárias ao projeto, instala Docker, Docker Compose e Git, clona o repositório do GitHub e executa a solução com Docker Compose.

A VM foi criada na região northcentralus e recebeu um IP público utilizado para acessar a aplicação externamente.

```
# criação automatizada (requer: az login)
bash scripts/azure-create-vm.sh

# passo a passo manual na VM
ssh azureuser@IP_DA_VM
curl -fsSL https://get.docker.com -o get-docker.sh && sudo sh get-docker.sh
git clone https://github.com/HenriqueRodriguesV/DevOpsGS.git
cd DevOpsGS
sudo docker compose up -d --build
sudo docker compose ps
```

PRINT 1 — Portal Azure: VM em execução

Tela 'Máquinas virtuais' com a VM (Status: Em execução, North Central US) e o IP público.

[inserir print aqui]

PRINT 2 — Teste de conectividade das portas

Terminal local com nc -vz IP_DA_VM 22 / 8080 / 8082 retornando 'succeeded'.

[inserir print aqui]

8 CONTEINERIZAÇÃO COM DOCKER

A aplicação foi containerizada utilizando Docker. O projeto possui um Dockerfile para a API Java Spring Boot e um Dockerfile específico para o banco H2. A orquestração dos containers foi feita com Docker Compose, permitindo subir a API e o banco com um único comando.

A solução foi executada em background com o comando `docker compose up -d --build`, atendendo ao requisito de manter o projeto rodando em segundo plano na VM. Os dois containers recebem o RM do representante no nome: `orbit-api-rm562917` e `orbit-db-rm562917`.

```
docker compose up -d --build    # sobe os containers em background
docker compose ps               # lista os containers em execução
docker compose logs orbit-api   # logs da aplicação
docker compose logs orbit-db    # logs do banco
```

PRINT 3 — docker compose ps

Containers `orbit-api-rm562917` e `orbit-db-rm562917` com status 'Up' e portas 8080/8082/9092 mapeadas.

[inserir print aqui]

PRINT 4 — Logs dos dois containers

Saída de `docker compose logs orbit-api` e `docker compose logs orbit-db`.

[inserir print aqui]

9 BANCO DE DADOS H2 E PERSISTÊNCIA

O banco escolhido para a entrega foi o H2 Database, executado em modo servidor TCP em um container separado da aplicação, permitindo que a API se conecte ao banco via TCP na rede orbit-net. Para garantir a persistência, foi configurado o volume nomeado orbit-h2-data, montado em /data.

A persistência foi validada criando um novo registro pela API, reiniciando os containers e consultando novamente os dados. Após o restart, o registro permaneceu salvo, comprovando que o volume nomeado estava funcionando corretamente. As consultas foram feitas com SELECT conectado diretamente no container do banco.

```
# dados de conexão H2 Console (porta 8082)
JDBC URL: jdbc:h2:tcp://localhost:9092/./orbitdb
User: sa      Password: (vazio)

# SHOW TABLES direto no container do banco
docker container exec -it orbit-db-rm562917 sh -c \
'java -cp h2.jar org.h2.tools.Shell -url jdbc:h2:tcp://localhost:9092/./orbitdb \
\
-user sa -password "" -sql "SHOW TABLES"'

# SELECT em duas tabelas (evidência de persistência)
... -sql "SELECT * FROM TB_ORBIT_PESSOA"
... -sql "SELECT * FROM TB_ORBIT_MISSAO"
```

PRINT 5 — Volume nomeado

docker volume ls | grep orbit mostrando orbit-h2-data.

[inserir print aqui]

PRINT 6 — SHOW TABLES + SELECT no container do banco

H2 Console (ou docker exec) exibindo as tabelas TB_ORBIT_* e os dados dos SELECT.

[inserir print aqui]

PRINT 7 — Persistência após restart

docker compose restart seguido de consulta mostrando que os dados continuam salvos.

[inserir print aqui]

10 TESTES DA API E CRUD

A API possui CRUD completo para os recursos de pessoas, missões, veículos, recursos e pontos de apoio. Os endpoints são protegidos por autenticação JWT: primeiro registra-se/realiza-se login para obter o token, que é então usado nas demais requisições. Os testes foram realizados externamente pelo IP público da VM, comprovando que a aplicação estava acessível pela internet e funcionando corretamente em nuvem.

Foram testadas operações de listagem, cadastro, atualização e remoção, atendendo aos métodos GET, POST, PUT e DELETE solicitados na atividade.

```
# 1) Registrar e obter token
curl -X POST http://IP_DA_VM:8080/api/auth/registrar \
-H "Content-Type: application/json" \
-d '{"nome":"Rep RM562917","email":"rep@orbit.com","senha":"orbit123"}'

TOKEN="COLE_O_TOKEN_AQUI"

# CREATE / READ / UPDATE / DELETE
curl -X POST http://IP_DA_VM:8080/api/pessoas -H "Authorization: Bearer $TOKEN"
...
curl http://IP_DA_VM:8080/api/pessoas -H "Authorization: Bearer $TOKEN"
curl -X PUT http://IP_DA_VM:8080/api/pessoas/1 -H "Authorization: Bearer $TOKEN" ...
curl -X DELETE http://IP_DA_VM:8080/api/pessoas/1 -H "Authorization: Bearer $TOKEN"
```



PRINT 8 — Swagger UI em nuvem

http://IP_DA_VM:8080/swagger-ui.html com os recursos (Pessoas, Missões, Veículos, Recursos, Pontos de Apoio) e GET/POST/PUT/DELETE.

[inserir print aqui]



PRINT 9 — Operações CRUD executadas

Respostas JSON do CREATE, READ (listagem), UPDATE e DELETE feitas pelo Swagger ou curl.

[inserir print aqui]

11 EVIDÊNCIAS DOS CONTAINERS

Conforme exigido, foi feito o acesso ao terminal dos dois containers com docker container exec, demonstrando o diretório atual (pwd), a estrutura de diretórios (ls -l) e o usuário conectado (whoami). Os containers executam com usuários não privilegiados: orbituser na aplicação e h2user no banco.

```
# Container da aplicação
docker container exec -it orbit-api-rm562917 pwd      # /app
docker container exec -it orbit-api-rm562917 ls -l
docker container exec -it orbit-api-rm562917 whoami   # orbituser

# Container do banco
docker container exec -it orbit-db-rm562917 pwd      # /h2
docker container exec -it orbit-db-rm562917 ls -l
docker container exec -it orbit-db-rm562917 whoami   # h2user
```

PRINT 10 — orbit-api: pwd / ls -l / whoami

Saída mostrando /app e o usuário orbituser (não-root).

[inserir print aqui]

PRINT 11 — orbit-db: pwd / ls -l / whoami

Saída mostrando /h2 e o usuário h2user (não-root).

[inserir print aqui]

12 LINKS DA ENTREGA

Os links abaixo devem ser utilizados para avaliação da entrega.

Item	Link
Repositório GitHub	https://github.com/HenriqueRodriguesV/DevOpsGS
Vídeo no YouTube	(preencher após gravação e publicação)

O link do vídeo deve ser preenchido somente após a gravação e publicação no YouTube.

13 REMOÇÃO DOS RECURSOS EM NUVEM

Após a conclusão dos testes, gravação do vídeo e coleta das evidências, os recursos criados na Azure devem ser removidos para evitar consumo desnecessário da assinatura. A remoção é feita por meio do script `azure-delete-resources.sh`, que exclui o Resource Group utilizado na entrega.

```
chmod +x scripts/azure-delete-resources.sh
./scripts/azure-delete-resources.sh
```



PRINT 12 — Exclusão dos recursos na Azure

Terminal com a remoção do Resource Group e/ou Portal Azure mostrando 'Não há máquinas virtuais para exibir'.

[inserir print aqui]

14 CONCLUSÃO

A entrega demonstrou a containerização e execução em nuvem de uma API Java Spring Boot com banco H2. O projeto foi executado em uma VM Linux na Azure, utilizando Docker e Docker Compose para subir a aplicação e o banco em containers separados, na mesma rede Docker e com persistência em volume nomeado.

A solução atendeu aos requisitos da atividade, incluindo CRUD completo, persistência em múltiplas tabelas relacionadas, volume nomeado, execução em background, containers rodando com usuários sem privilégios administrativos, exposição de portas, variáveis de ambiente, scripts Azure CLI, documentação no GitHub e validação externa da API.

Além da parte técnica, o projeto possui uma proposta de negócio coerente com o tema da Global Solution 2026/1, servindo como base para uma futura plataforma de gestão de operações espaciais conectada a problemas reais na Terra.

REFERÊNCIAS

FIAP. Conteúdo acadêmico da disciplina DevOps Tools & Cloud Computing. São Paulo: FIAP, 2026.

MICROSOFT. Azure CLI Documentation. Disponível em: <https://learn.microsoft.com/cli/azure/>. Acesso em: 08 jun. 2026.

DOCKER. Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em: 08 jun. 2026.

SPRING. Spring Boot Reference Documentation. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 08 jun. 2026.

H2 DATABASE. H2 Database Engine Documentation. Disponível em: <https://www.h2database.com/>. Acesso em: 08 jun. 2026.

GITHUB. Repositório do projeto ORBIT API. Disponível em: <https://github.com/HenriqueRodriguesV/DevOpsGS>. Acesso em: 08 jun. 2026.